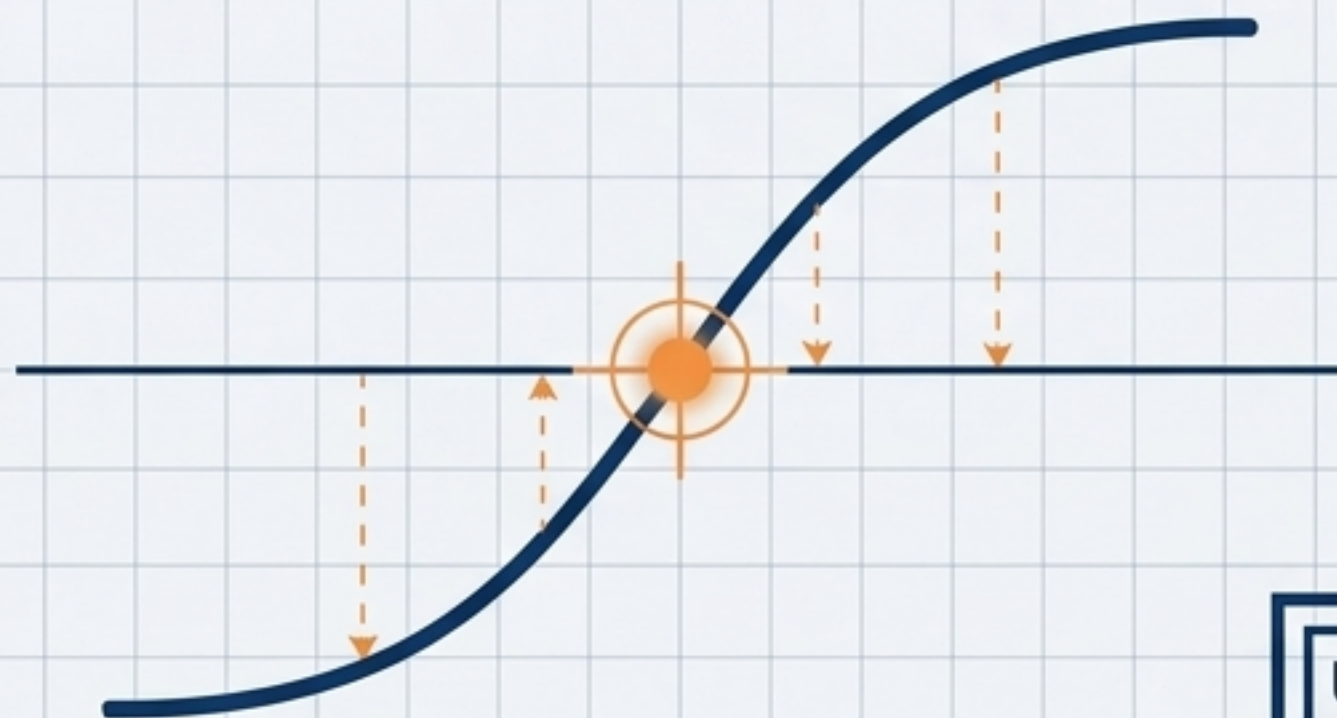


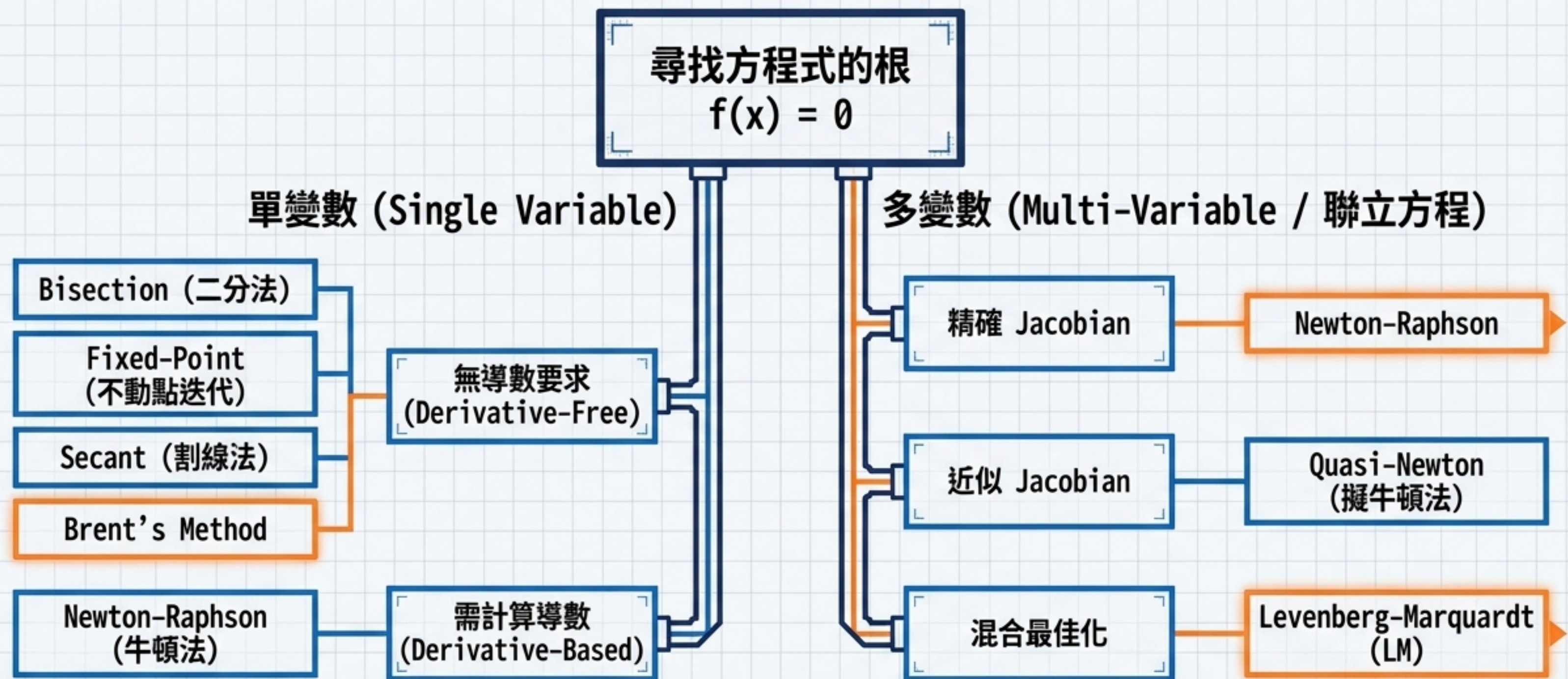
數值方法理論基礎

破解非線性方程式的工程演算法



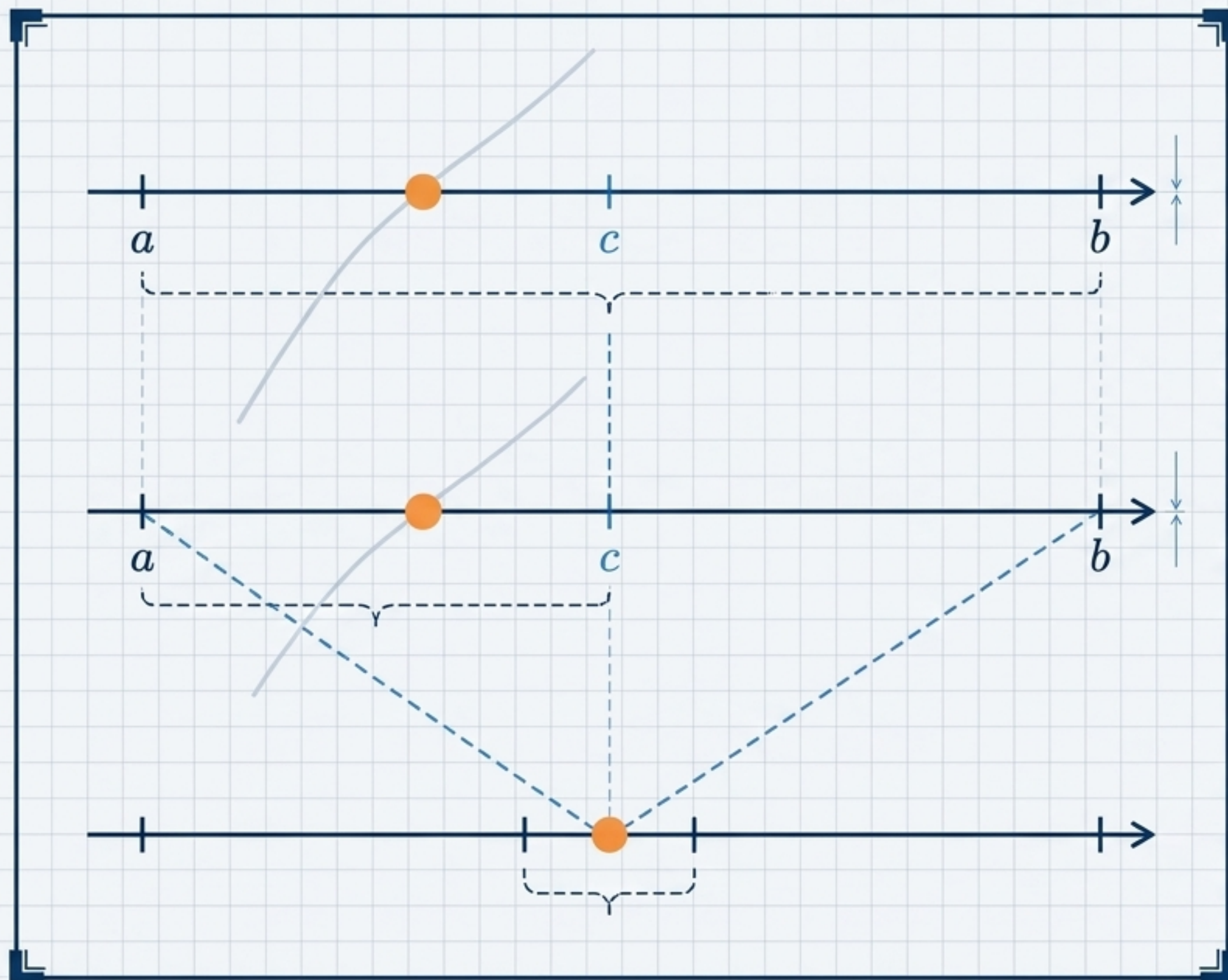
UNIT:	07
TOPIC:	Nonlinear Equations
MODULE:	scipy.optimize

演算法分類地圖 (The Algorithm Taxonomy)



選擇演算法的本質，是在穩定性 (Robustness) 與 收斂速度 (Speed) 之間尋找最佳平衡。

Bisection 法 (二分法)：絕對可靠的基礎工具



定理 (Intermediate Value Theorem)

若 $f(a) \cdot f(b) < 0$ ，則區間 $[a, b]$ 內必存在至少一個根。

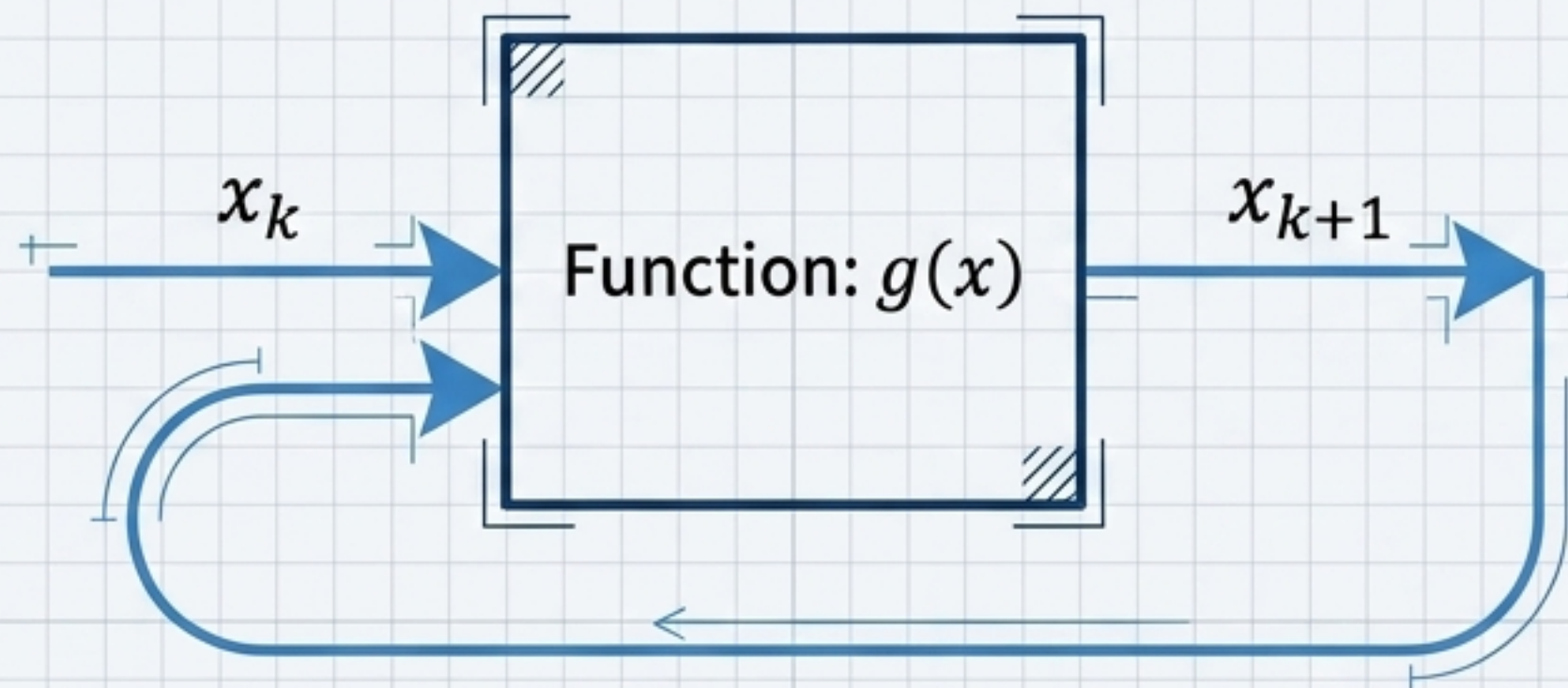
機制：每次迭代取中點 $c = \frac{a + b}{2}$ ，將搜尋區間縮減一半。

✓ 優點：保證收斂（只要初始區間正確）、不需計算導數。適合處理不連續可微的函數。

✗ 缺點：收斂速度極慢（線性收斂）、需事先給定包含根的上下界。

化工應用：求解 Van der Waals 氣相根時，需要進行高達 **28次** 迭代才能達到機器精度。

不動點迭代法 (Fixed-Point)：將求解轉化為回饋迴路



機制 (Mechanism)

1. 改寫方程式：將 $f(x) = 0$ 轉換為 $x = g(x)$ 的形式。
2. 建立迴路： $x_{k+1} = g(x_k)$

實作簡單，但速度偏慢且不穩定，通常僅用於特定類型的簡單方程式。



收斂條件限制

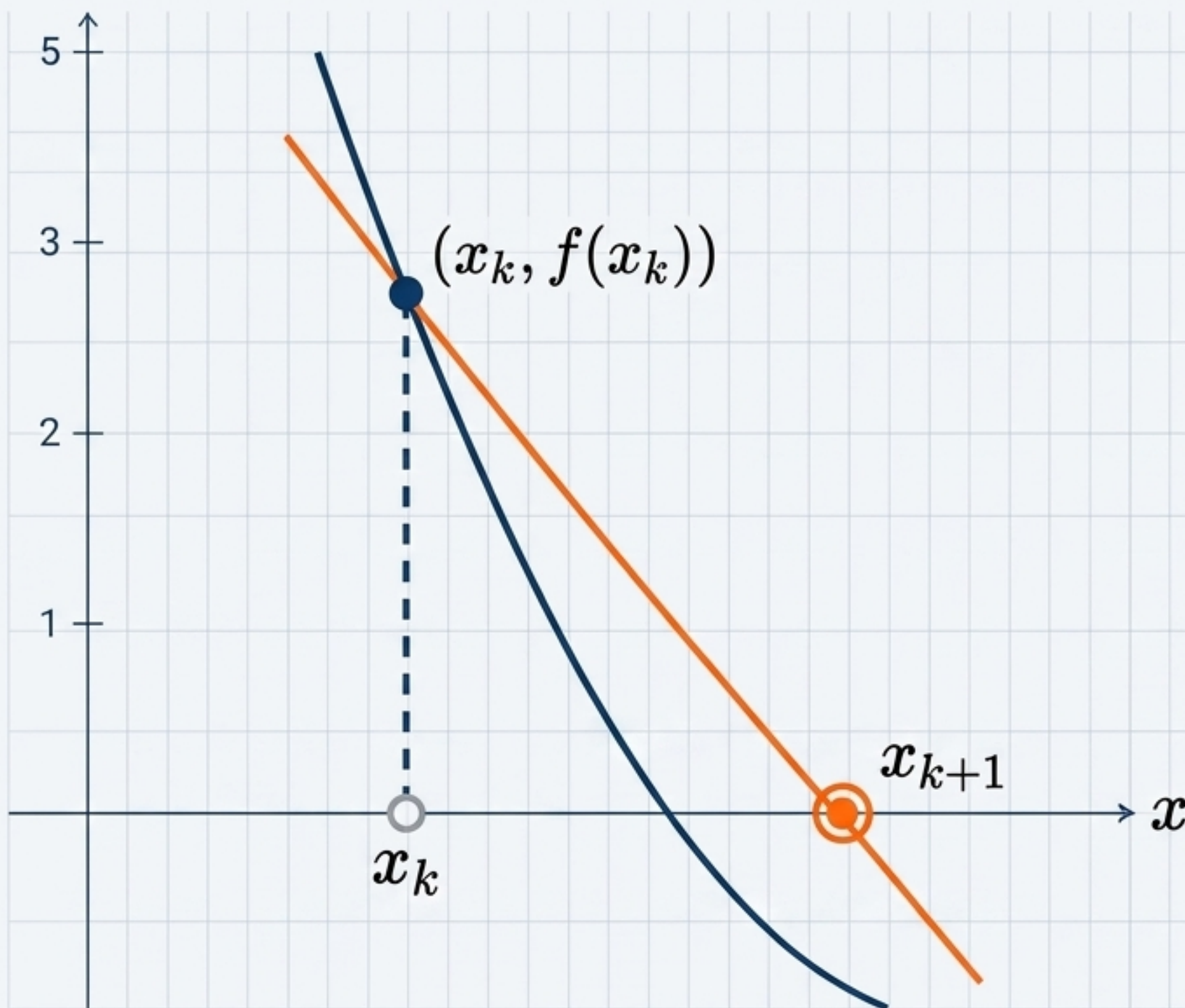
$$|g'(x)| < 1$$

若在根附近的導數絕對值大於 1，迭代序列將會發散 (Diverge)。收斂性高度依賴 $g(x)$ 的形式選擇。

Newton-Raphson 法：精準的高速切線



二次收斂 (Quadratic Convergence) - 在根附近，每次迭代有效位數翻倍。速度極快。



$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

多變數升級路徑 (Multi-Variable Upgrade Path)

當擴展至多變數系統 $F(x) = 0$ 時，導數 $f'(x)$ 升級為 Jacobian 矩陣 J ：

$$x_{k+1} = x_k - J^{-1}(x_k)F(x_k)$$

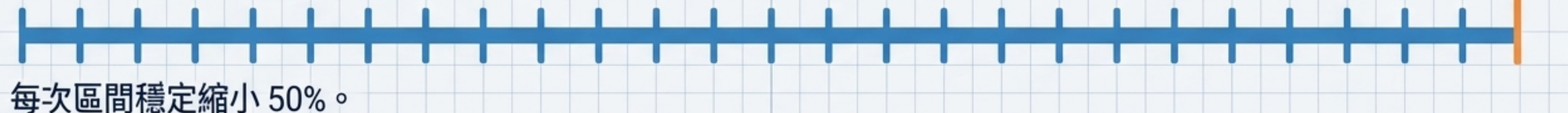
效能對決：穩定性 vs. 極致速度

案例：求解 CO2 Van der Waals 方程式 (Target Root: $V \approx 0.364$ L/mol)

Performance Race

Bisection 法 (線性收斂)

28 次迭代 (到達機器精度)



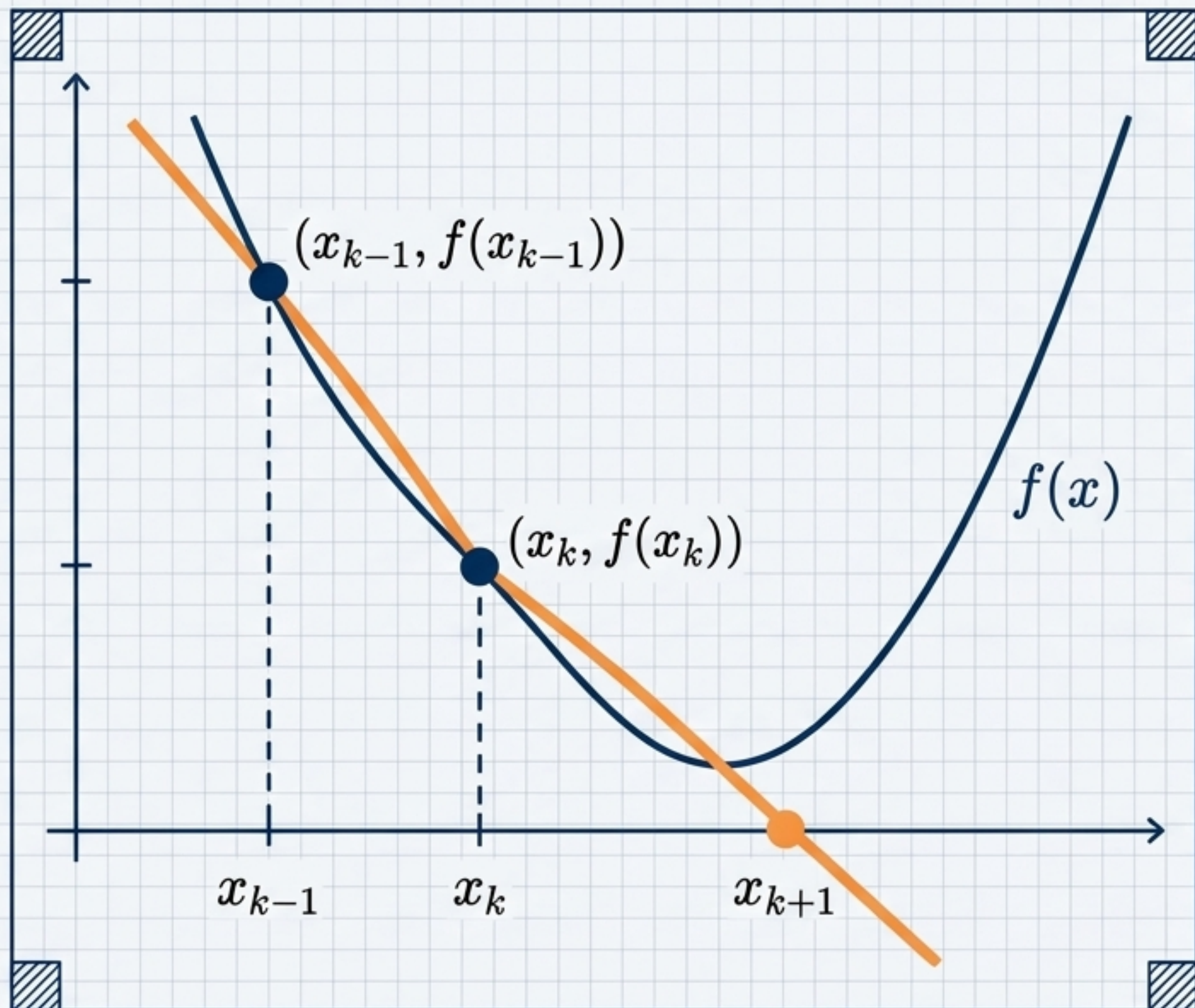
Newton-Raphson 法 (二次收斂)

僅需 4 次迭代



注意：Newton 法速度驚人，但代價是必須具備數學解析式來計算導數 $f'(x)$ ，且對起始猜測值 x_0 極度敏感。

Secant 法 (割線法)：免計算導數的捷徑



差分近似 (Finite Difference)

使用差分近似導數，完美避開繁瑣的解析微分計算。

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

權衡與代價

➡ **速度**：超線性收斂 (Superlinear)。介於二分法與牛頓法之間。

⚠ **代價**：需要提供兩個起始猜測值 (x_k, x_{k-1}) ，且在極端情況下仍有發散風險。

Brent's Method：單變數求解的終極混血兒

結合穩定性與速度的工程奇蹟

Block 3 (微調)：IQI (反二次插值)

在函數平滑處進行極致逼近。



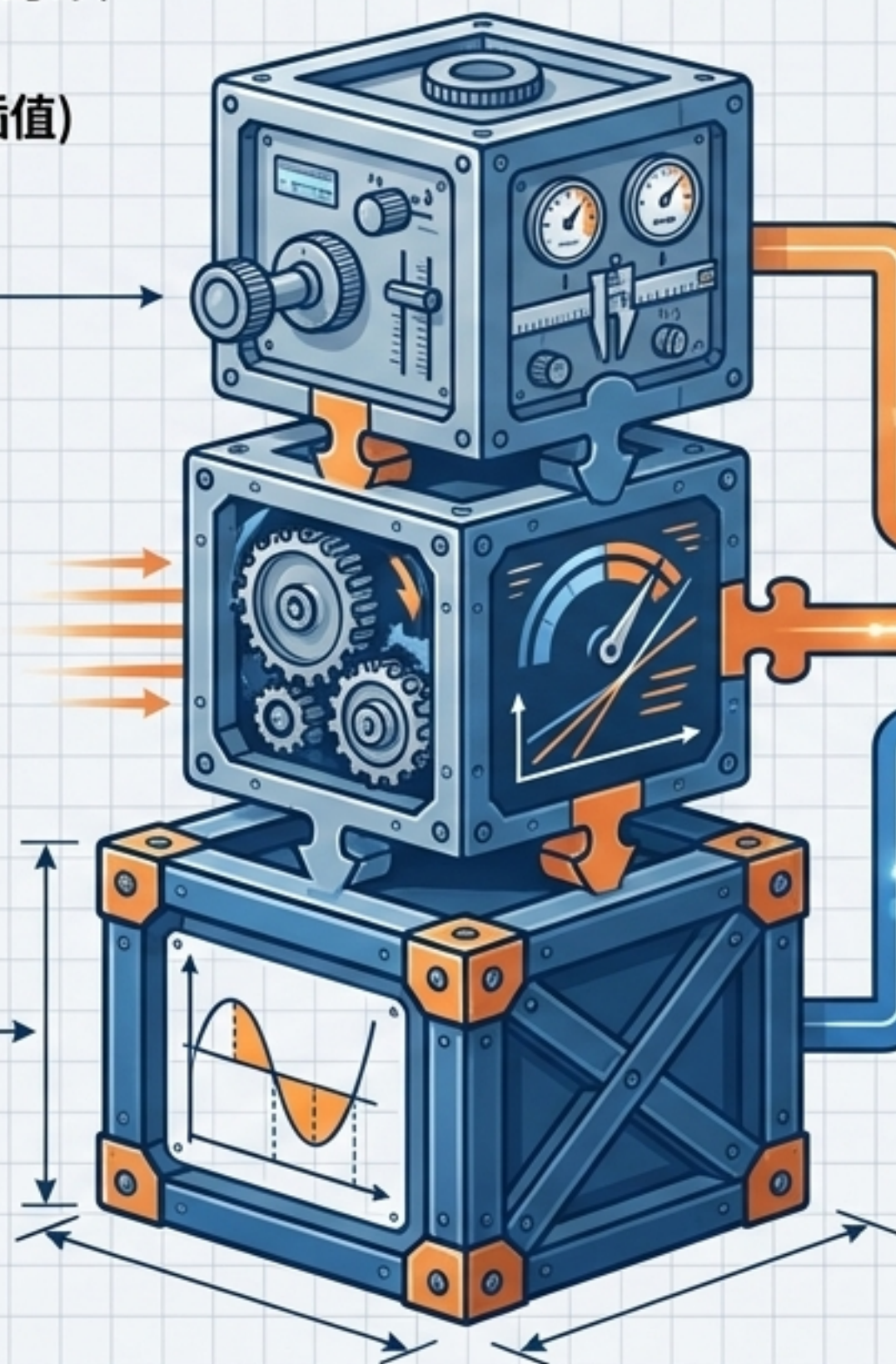
Block 2 (加速器)：Secant 法

提供無導數要求的超線性速度。



Block 1 (基底)：Bisection 法

提供絕對的穩定性與保證收斂。



單變數問題的終極解決方案



Industry Standard
SciPy 預設首選

```
scipy.optimize.root_scalar  
(method='brentq')
```

實務上單變數問題的最佳解，
完美兼顧了二分法的防呆機
制與割線法的高效率。

擬牛頓法 (Quasi-Newton)：突破多變數運算瓶頸

The Bottleneck

$$J \begin{bmatrix} v_{k+1} & v_{k+1} & v_{k+1} & v_{k+1} & \dots & v_{k+1} & v_{k+1} & v_{k+1} \\ v_{k+1} & v_{k+1} & v_{k+1} & v_{k+1} & \dots & v_{k+1} & v_{k+1} & v_{k+1} \\ v_{k+1} & v_{k+1} & v_{k+1} & v_{k+1} & \dots & v_{k+1} & v_{k+1} & v_{k+1} \\ v_{k+1} & v_{k+1} & v_{k+1} & v_{k+1} & \dots & v_{k+1} & v_{k+1} & v_{k+1} \\ v_{k+1} & v_{k+1} & v_{k+1} & v_{k+1} & \dots & v_{k+1} & v_{k+1} & v_{k+1} \\ v_{k+1} & v_{k+1} & v_{k+1} & v_{k+1} & \dots & v_{k+1} & v_{k+1} & v_{k+1} \end{bmatrix}$$

$J (N \times N \text{ Calculations})$

問題：在大型化工系統中，每次迭代重新計算完整的 Jacobian 矩陣會消耗天文數字的資源。

The Solution

$$J_k + \frac{\begin{bmatrix} 1 \end{bmatrix}}{\begin{bmatrix} -Jx \end{bmatrix}}$$

解法：利用前一次迭代的資訊逐步「逼近」Jacobian，而非從頭計算。

Broyden 方法
(Rank-1 Update)

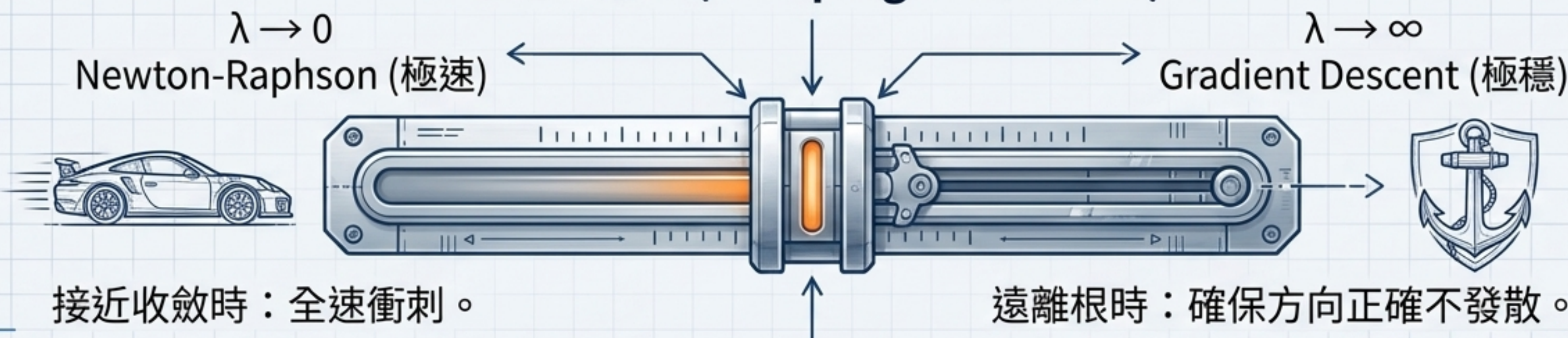
$$J_{k+1} \approx J_k + \left[\frac{(\Delta F - J_k \Delta x) \Delta x^T}{\Delta x^T \Delta x} \right]$$

優勢：大幅降低計算成本，是大型系統 (scipy.optimize.root(method='broyden1')) 的首選。

Levenberg-Marquardt (LM) 演算法：靈活的穩健引擎

核心概念：結合了牛頓法的速度與梯度下降法的穩定性。

阻尼參數 λ (Damping Parameter)



應用場景 (Best Use Cases)

- 對起始猜測值極不敏感。
- 特別適合處理過確定系統 (Overdetermined systems，方程式數量 > 未知數數量)。

SciPy 實作

```
root(method='lm')  
least_squares()
```


數值求解工具箱：工程師決策矩陣 (Decision Matrix)

方法名稱 (Method)	收斂速度 (Speed)	需要導數？ (Derivatives?)	適用維度 (Dimension)	最佳應用情境 (Best Case)
Bisection	線性 (Linear)	⊗	單變數	已知根的上下界，求絕對穩定。
Newton-Raphson	二次 (Quadratic)	✓	單/多變數	函數平滑且易計算 Jacobian 時。
Brent's	混合 (Hybrid)	⊗	單變數	單變數系統的 [預設首選] 。
Quasi-Newton	超線性	⊗ (近似)	多變數	大型稀疏系統，節省算力。
Levenberg-Marquardt	混合 (Hybrid)	✓/近似	多變數	過確定系統或起始點極度不確定時。

實務守則：在 SciPy 中，單變數問題一律優先嘗試 `brentq`；多變數問題優先使用 `hybr` 或 `lm`。